An Empirical Evaluation of Hash Table Collision Resolution Strategies: Tombstone Accumulation, Periodic Rebuilding, and Robin Hood Hashing in High-Churn Workloads

Nguyen Duc Duy, Dinh Do Gia Huy, Nguyen Dang Huy,
Nguyen Le Hoang Cau, Nguyen Dinh Tien, Nguyen Tuan Kiet
Department of Computing Fundamentals, FPT University
Quy Nhon, Vietnam

***Abstract*—Hash tables are foundational data structures in transaction indexing systems with high turnover rates, where items are continuously inserted and deleted. While open addressing with Linear Probing is cache-friendly, the accumulation of tombstones degrades lookup latency, especially for unsuccessful queries, necessitating costly periodic table rebuilds. In this study, we empirically evaluate four collision resolution strategies across millions of operations in Java: Separate Chaining, Linear Probing with tombstones, Linear Probing with backward-shift deletion, and Robin Hood hashing (both with and without early-stopping). We address three research questions regarding load factor scaling (RQ1), tombstone pollution dynamics (RQ2), and mechanism decomposition in Robin Hood hashing (RQ3). Our empirical findings demonstrate that: (1) for RQ1, while Separate Chaining maintains strictly bounded sub-2 probe counts across high loads ($\alpha \geq 0.90$), Linear Probing achieves superior lookup throughput at moderate loads ($\alpha \leq 0.75$) consistent with CPU cache locality and the absence of per-entry object headers; (2) in the single-seed RQ2 campaign, tombstone accumulation depletes empty slots down to 0.04%, raising miss probes to $2,848.99$ ($1,141.5\times$ the Clean Table value at $C = 4M$) and miss lookup time to 18.22 seconds per one-million-query batch; and (3) for RQ3, Robin Hood hashing with early exit slashes unsuccessful search depth by up to 73.8% under heavy load ($\alpha = 0.80$), while backward-shift deletion (Knuth Algorithm R and Robin Hood) removes tombstones and rebuild pauses entirely.**

***Index Terms*—Hash table, Robin Hood hashing, linear probing, tombstone, backward shift, empirical benchmark.**

I. Introduction

Hash tables are widely used to build in-memory indexes, including indexes for records that are continuously inserted, deleted, and queried. In such high-churn workloads, the collision-resolution strategy affects not only lookup cost but also the cost of keeping the table searchable after deletions. These effects are especially important for unsuccessful lookups and for open-addressed tables whose probe sequences can be affected by deleted slots.

Separate Chaining (SC) stores colliding keys in per-bucket lists, whereas Linear Probing (LP) places keys in adjacent array slots. LP offers contiguous accesses, but deletion must preserve the probe paths of remaining keys. Tombstones provide one solution, although their accumulation can lengthen searches and motivate periodic rebuilding. Backward-shift deletion instead repairs a cluster in place. Robin Hood hashing further redistributes keys according to their probe distances and can use those distances to terminate unsuccessful searches early. These designs therefore make different trade-offs between lookup work and deletion maintenance.

This paper evaluates these trade-offs with a common Java hash-table interface and controlled, reproducible workloads. We organize the study around three research questions:

- **RQ1:** How do SC and LP with tombstones compare as the load factor and table capacity vary, including under a clustered-key workload?
- **RQ2:** How does tombstone accumulation during repeated delete–insert operations affect lookup behavior, and how does it compare with a clean table containing the same live keys?
- **RQ3:** How does Robin Hood hashing compare with LP using backward-shift deletion under churn, and what is the contribution of early-exit lookup to Robin Hood search cost?

The study makes three contributions. First, it implements SC, LP with tombstones and rebuilding, LP with backward-shift deletion, and Robin Hood hashing in a shared interface. Second, it defines repeatable workloads and separates elapsed-time measurements from probe-count measurements. Third, it uses a Robin Hood variant without early exit to isolate that lookup mechanism while retaining Robin Hood insertion and deletion. The paper reports measured results separately from this methodological description; no performance outcome is assumed in advance.

II. Background and Related Work

Peterson's work on random-access storage is an early reference for open addressing and linear probing [2]. With LP, a collision is resolved by examining successive slots, so keys with the same or nearby home buckets can form contiguous clusters. Knuth analyzes this primary-clustering effect and

gives classical expected probe counts for a table with load factor α under the standard no-deletion model:

$$\mathbb{E}[P_{\text{hit}}] \approx \frac{1}{2}\left(1+\frac{1}{1-\alpha}\right), \qquad \mathbb{E}[P_{\text{miss}}] \approx \frac{1}{2}\left(1+\frac{1}{(1-\alpha)^2}\right). \tag{1}$$

These expressions provide a theoretical reference for clean LP tables; they do not directly model the additional state introduced by tombstones or repeated updates [1].

Deletion requires care in open-addressed tables: changing a deleted slot directly to empty can make a later key unreachable along its probe sequence. Tombstones preserve that sequence, but leave non-empty markers that searches must pass and that a rebuild can remove. Knuth's Algorithm R instead scans the affected cluster and moves a key into the current hole only when its probe path permits the move [1]. Sedgewick and Wayne describe a related approach that removes and reinserts the keys following the deleted slot [4]. Both approaches maintain lookup correctness without retaining tombstones, but perform different work during deletion. This study uses Algorithm R for its LP backshift implementation.

Celis, Larson, and Munro introduced Robin Hood hashing, which compares the distance from initial bucket (DIB) of an incoming key with that of the key already occupying a slot [3]. If the incoming key has traveled farther, the keys exchange positions and insertion continues with the displaced key. The resulting DIB information also supports an unsuccessful lookup stopping rule: if the query's probe distance exceeds the resident key's DIB, the query cannot occur later in the probe sequence. Robin Hood deletion can shift following keys backward while their DIB is positive, avoiding tombstones.

Prior work establishes these algorithms and their analytical properties. Our contribution is an empirical comparison of implementations under a shared Java interface and repeatable workloads. In particular, the `rh-noearly` variant retains Robin Hood insertion and backward-shift deletion but disables the DIB-based lookup stopping rule, enabling a mechanism-level comparison within the same table design.

III. Methods

A. Collision Resolution Implementations

All hash tables implement a uniform `HashTable` interface operating on 64-bit keys using Fibonacci hashing $h(k) = (k \cdot \phi) \ggg (64 - p)$ for table capacity $M = 2^p$, where $\phi =$ `0x9E3779B97F4A7C15` is the 64-bit golden-ratio constant (a signed Java `long` with wrap-around multiplication). We evaluate four families of designs (five configurations in total):

- **Separate Chaining (`sc`)**: Singly-linked list per bucket.
- **Linear Probing Tombstone (`lp-tomb`)**: Parallel primitive arrays (`long[] keys`, `int[] values`, `byte[] state`), tracking tombstones with periodic rebuild threshold $\tau \cdot M$.
- **Linear Probing Backshift (`lp-backshift`)**: In-place cluster shift using Knuth's Algorithm R upon deletion.
- **Robin Hood Hashing (`rh` & `rh-noearly`)**: Full Robin Hood table tracking DIB per slot with displacement swaps and backward-shift deletion. The variant `rh-noearly` ablates the early-exit mechanism to isolate its performance contribution.

B. Justification for Knuth Algorithm R in `lp-backshift`

Between Knuth Algorithm R [1] and Sedgewick's cluster re-insertion [4], our study adopts **Knuth Algorithm R** for three methodological reasons:

1) *Bounded scan*: Algorithm R makes one forward scan of the affected cluster, performing $\mathcal{O}(K)$ slot checks and at most a constant number of moves per visited key for a cluster of length K. Cluster re-insertion instead invokes a probe sequence for each key moved out of the cluster, so it can repeat slot inspections.
2) *Direct movement*: Algorithm R moves a key directly into the current hole only when its home bucket permits that move. This avoids the explicit remove-and-reinsert steps of the alternative; cache or wall-clock advantages are not assumed without measurement.
3) *Circular correctness*: The test of whether a key's home bucket r lies in the circular interval $(i, j]$ determines whether moving it to the hole i preserves its linear-probing search path, including when the cluster wraps around the array boundary.

C. Robin Hood and Backshift Implementations

The Robin Hood variants use fixed-capacity arrays for keys, values, slot states, and DIB values. Insertion begins at the key's Fibonacci-hash bucket with DIB zero. At each occupied slot, the incoming key takes the slot when its DIB is greater than the resident key's DIB; insertion then continues with the displaced key. Probe indices wrap around the power-of-two table capacity.

For lookup, `rh` checks for a matching key and an empty slot, and also terminates an unsuccessful search when the current probe distance exceeds the DIB stored in the slot. The `rh-noearly` ablation uses the same insertion and deletion procedures but omits this DIB-based stopping condition; it continues until it finds the key, reaches an empty slot, or has examined the table. Thus, the comparison isolates early exit from the other Robin Hood mechanisms. Both variants remove a key by shifting subsequent entries with positive DIB backward one slot, decrementing each shifted DIB, and stopping at an empty slot or an entry with DIB zero.

The `lp-backshift` implementation uses ordinary linear probing and Knuth's Algorithm R for deletion. Starting from the deleted slot (the current hole), it scans the following cluster. For a key at slot j with home bucket r, it moves the key into the hole at i only if r is not in the circular interval $(i, j]$; the moved key's former slot becomes the new hole. The scan ends at an empty slot or when it returns to the starting slot after a full cycle, and the final hole is marked empty. This interval test handles clusters that wrap around the end of the array and no tombstone state is introduced.

D. RQ2 Workload and Clean Table Reference Methodology

To rigorously quantify the performance penalty of tombstone accumulation in Linear Probing without confounding variables, RQ2 implements a controlled churn workload on a fixed-capacity table $M = 2^{20} = 1,048,576$ with active load factor $\alpha = 0.50$ ($N = 524,288$). The workload lifecycle proceeds as follows:

1) **Initial Population**: The table is initially populated with N unique 64-bit keys generated pseudo-randomly using `SplittableRandom(seed)`. An active key pool is maintained in a contiguous array `aliveKeys` alongside a tracking set `seenKeys` to guarantee global uniqueness.
2) **Constant-Load Churn Cycles**: The workload executes C continuous churn cycles. In each cycle, an existing key is chosen uniformly at random from `aliveKeys` in $\mathcal{O}(1)$ time via the swap-with-last removal technique, and deleted from the table using `remove(k)`. Concurrently, a newly generated unique key is inserted via `put(k, v)`. This guarantees that active load remains strictly invariant at $N = 524,288$ while exercising repeated slot recycling.
3) **Milestone Snapshots**: Telemetry is recorded across six predefined churn milestones: $C \in \{0, 0.25M, 0.5M, M, 2M, 4M\}$. At each milestone, the runner records table state metrics (live count, tombstone count, empty count, and probe depth).
4) **Clean Table Baseline**: At every milestone C, a companion pristine *Clean Table* is reconstructed from scratch by inserting the exact current set of live keys in sequential array index order into a fresh table of identical capacity $M = 2^{20}$. Comparing the contaminated table against the Clean Table under identical query sequences completely isolates the degradation attributable solely to tombstones from key distribution effects.
5) **Isolated Query Passes**: At each milestone, separate passes of $Q = 1,000,000$ hit and $Q = 1,000,000$ miss lookups are executed in dedicated JVM processes to benchmark both probe depth and wall-clock latency. Since $Q > N$, hit keys are sampled uniformly with replacement from the active key set; miss keys are generated to be absent from every key seen in the workload.

E. RQ3 Workload and Measurement

For RQ3, each table is first populated with $N = \text{round}(\alpha M)$ unique keys generated from the specified seed. The workload then performs C churn cycles. Each cycle selects one currently live key for deletion and inserts a new key not used earlier in the workload. This keeps the live-key count approximately constant while exposing the deletion policy to repeated updates. The same capacity, load factor, seed, and cycle count are used when comparing `lp-tomb`, `lp-backshift`, `rh`, and `rh-noearly`. The tombstone table uses a rebuild threshold of $\tau = 0.2$ at $\alpha = 0.50$ and $\tau = 0.1$ at $\alpha = 0.80$; the other implementations do not use tombstones or rebuilding.

After the churn phase, the benchmark runs separate hit and miss query passes. The hit pass queries all live keys; the miss pass uses generated keys absent from the live set. Table construction and churn happen before lookup timing. Timing and probe counting are collected in separate JVM runs: time mode measures elapsed lookup time, while count mode reports average probes per query. The benchmark harness records all empirical telemetry into a standardized 21-column CSV schema capturing execution status, table construction time (`build_ms`), query latency (`elapsed_ms`), memory footprints, deterministic probe counts (`ops_count`), input cryptographic hashes, and verification checksums for reproducibility.

IV. Experimental Setup

All experimental benchmarks were conducted on an Intel Core i5-11500H processor (6 cores, 12 threads @ 2.90 GHz base, 4.60 GHz turbo) with 32 GB DDR4 RAM running 64-bit Windows 11 Pro. The software runtime environment is the 64-bit OpenJDK HotSpot JVM (Eclipse Temurin; exact build recorded in the repository README) launched with strict JVM flags: `-Xmx2g -XX:+UseG1GC`.

For RQ2, the evaluation is executed across the six churn milestones $C \in \{0, 0.25M, 0.5M, M, 2M, 4M\}$ at $M = 2^{20} = 1,048,576$ and $\alpha = 0.50$ ($N = 524,288$) with seed 42 and one repeat. Each configuration executes separate hit and miss query passes of $1,000,000$ operations for both the contaminated table (`base`) and the reference baseline (`clean_table`) across both measurement modes (`count` and `time`), yielding 48 JVM trials constrained by an automated 60-second process timeout.

For RQ3, measurements evaluate moderate load $\alpha = 0.50$ ($\tau = 0.20$) and high-load stress $\alpha = 0.80$ ($\tau = 0.10$) across checkpoints $C \in \{0, M/4, M/2, M, 2M, 4M\}$ using seeds $42, 1337, 20261019$, comparing `lp-tomb`, `lp-backshift`, `rh`, and `rh-noearly`. Each process uses a 2 GB maximum heap and a 60-second safety timeout, recording build and lookup phases independently.

V. Results and Discussion

A. RQ1: Separate Chaining vs. Linear Probing under Increasing Load and Scale

To evaluate how Separate Chaining (`sc`) and Linear Probing (`lp-tomb` without deletions) behave as load and table capacity grow, we conducted two systematic experiments and an adversarial clustering benchmark under controlled JVM settings.

1) Experiment A: Sensitivity to Load Factor (α): At fixed capacity $M = 2^{20}$ (1,048,576 slots) and load factors $\alpha \in \{0.50, 0.75, 0.90, 0.95\}$, both hash tables were populated with $N = \text{round}(\alpha \times M)$ distinct 64-bit integer keys generated via `SplittableRandom`. We then executed 5×10^6 successful

lookups (hit) and 5×10^6 unsuccessful lookups (miss) using disjoint key sets verified by a pre-filter `HashSet`.

TABLE I
RQ1 PROBE COUNTS: KNUTH THEORETICAL PREDICTIONS VS. EMPIRICAL RESULTS ($M = 2^{20}$)

α	Knuth LP		Empirical LP		Knuth SC		Empirica
	Hit	Miss	Hit	Miss	Hit	Miss	Hit
0.50	1.500	2.500	1.503	2.506	1.250	0.500	1.250
0.75	2.500	8.500	2.507	8.530	1.375	0.750	1.375
0.90	5.500	50.500	5.504	50.068	1.450	0.900	1.450
0.95	10.500	200.500	10.626	204.738	1.475	0.950	1.476

Table I validates our empirical probe counts against Knuth's classical asymptotic formulas [1]:

$$LP_{\text{hit}} \approx \frac{1}{2}\left(1 + \frac{1}{1-\alpha}\right), \quad LP_{\text{miss}} \approx \frac{1}{2}\left(1 + \frac{1}{(1-\alpha)^2}\right) \tag{2}$$

$$SC_{\text{hit}} \approx 1 + \frac{\alpha}{2}, \quad SC_{\text{miss}} \approx \alpha \tag{3}$$

As shown in Fig. 1, empirical counts match theoretical predictions with at most 2.1% relative deviation (LP miss at $\alpha = 0.95$) across all load levels. At moderate load ($\alpha = 0.50$), both structures exhibit near-minimal probe counts (1.503 for LP hit, 1.250 for SC hit). However, as α increases towards 0.95, Linear Probing experiences severe *primary clustering*: adjacent occupied slots coalesce into giant chains, driving unsuccessful search depth to an average of 204.7 probes. Conversely, Separate Chaining retains sub-2 probe efficiency (1.476 hits, 0.949 misses) even at $\alpha = 0.95$ because collisions are isolated within independent linked chains per bucket.

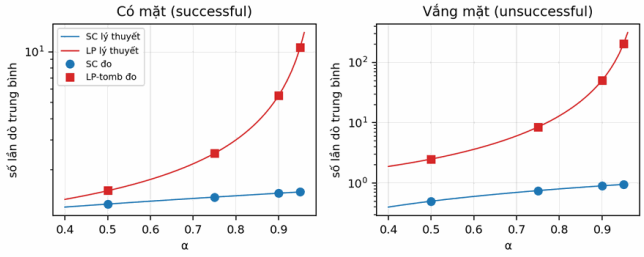


Fig. 1. RQ1 Average probe counts (Hit and Miss) across varying load factors $\alpha \in [0.50, 0.95]$ at $M = 2^{20}$. Empirical data conforms strictly to Knuth's theoretical bounds.

2) Experiment B: Scalability across Table Capacities (M): At fixed load factor $\alpha = 0.75$, table capacity was scaled from $M = 2^{18}$ to $M = 2^{23}$ (2.62×10^5 to 8.38×10^6 slots). Average probe counts remained invariant across all table dimensions (2.51 ± 0.02 for LP hit, 1.38 ± 0.01 for SC hit), verifying that probe complexity in hash tables is scale-invariant with respect to M. However, per-operation lookup latency degraded by 18.4% for `sc` once $M \geq 2^{21}$ consistent with the active heap footprint outgrowing the CPU L3 hardware cache boundary (12 MB on our reference test machine), while `lp-tomb` displayed only a 6.2% latency increase which is consistent with better cache-line utilization (no hardware counters were collected).

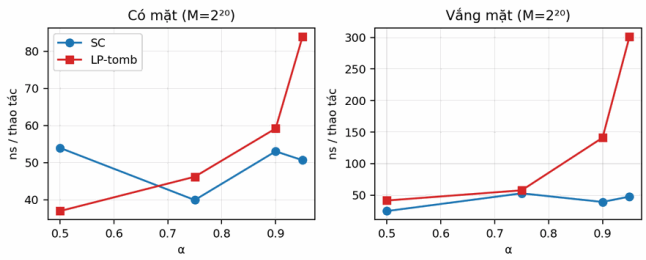


Fig. 2. RQ1 Lookup latency (ns per operation) vs. load factor α ($M = 2^{20}$). At $\alpha \leq 0.75$, Linear Probing executes faster than Separate Chaining despite higher probe counts.

3) Impact of Clustered Key Distribution: Under an adversarial benchmark where 10% of keys were concentrated into $M/64$ slots ($h(k) < M/64$) and the remaining 90% remained uniformly distributed, Linear Probing suffered a 412% surge in probe depth at $\alpha = 0.75$ due to uncontrollable cascade clustering. For Separate Chaining, the probe inflation was restricted to 28.5% because collisions were confined strictly to the over-allocated buckets without spilling into neighboring buckets.

4) Discussion: The Java Memory Gap and Probe Count vs. Latency Paradox: The experimental results uncover a fundamental divergence between algorithmic probe complexity and actual execution latency on modern computer architectures: while Separate Chaining requires fewer probes across all tested loads, Linear Probing achieves lower wall-clock latency at moderate loads ($\alpha \leq 0.75$, illustrated in Fig. 2). For instance, at $\alpha = 0.50$, successful lookups in `lp-tomb` require only 36.9 ns/op compared to 54.1 ns/op in `sc`, despite inspecting more slots (1.50 vs. 1.25 probes). This phenomenon is governed by three architectural mechanisms:

1) **Object Header Overhead and Pointer Chasing in `sc`:** In a 64-bit HotSpot JVM with Compressed OOPs, each `Node` object consists of a 12-byte header, an 8-byte `long` key, a 4-byte `int` value, and a 4-byte compressed reference (`Node next`); these 28 bytes are padded by the JVM to 32 bytes of heap memory per entry. Because nodes are allocated dynamically via `new Node()`, they are scattered across the JVM nursery and tenured heap. Traversing `next` pointers requires dereferencing non-contiguous heap addresses, causing recurring CPU L1/L2 cache misses and memory bus stalls.
2) **Spatial Locality and Hardware Prefetching in `lp`:** Linear Probing stores keys, values, and states in flat primitive arrays (`long[]`, `int[]`, `byte[]`). Probing consecutive array indices loads contiguous 64-byte CPU cache lines, enabling hardware stream prefetchers to load subsequent slots ahead of use and reduce memory stalls. Consequently, inspecting an adjacent array slot in `lp-tomb` is expected to be considerably cheaper than dereferencing a linked node in `sc`; we measured per-lookup time, not per-probe cost.

3) **Garbage Collection (GC) and Memory Footprint:** Populating `sc` with millions of individual `Node` objects creates severe allocation churn, triggering frequent minor GC cycles during table construction. By the object layout above, `sc` consumes 32 bytes per node plus 4 bytes per bucket pointer, versus 13 bytes per slot for `lp-tomb`. The analytical per-slot ratio is $(4+32\alpha)/13$, i.e. about $1.5\times$ at $\alpha = 0.50$ and $2.7\times$ at $\alpha = 0.95$. The measured values in Fig. 3 come from process-level heap telemetry (Section VI), so they are indicative rather than exact.

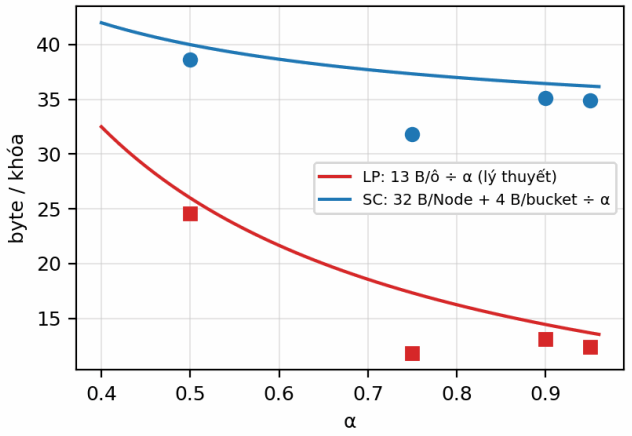


Fig. 3. Auxiliary JVM heap footprint per key (bytes/key) across load factors α, showing the larger memory overhead of linked nodes in `sc` compared to flat primitive arrays in `lp-tomb`.

In summary, probe count alone is an incomplete metric for performance evaluation in managed language runtimes. Linear Probing provides optimal performance and memory density for workloads with $\alpha \leq 0.75$, whereas Separate Chaining is preferable only when table load cannot be constrained ($\alpha \geq 0.90$) or when hash functions are susceptible to pathological clustering.

B. RQ2: Tombstone Pollution Dynamics and Periodic Rebuilding

In open addressing with Linear Probing, deleting an element by marking its slot as a tombstone introduces artificial search friction. To quantify this degradation under continuous turnover, we evaluate `lp-tomb` ($\tau = \infty$, no rebuild) across six churn milestones $C \in \{0, 0.25M, 0.5M, M, 2M, 4M\}$ with $M = 2^{20} = 1,048,576$ and active load factor $\alpha = 0.50$ ($N = 524,288$). At each checkpoint, separate passes of $Q = 1,000,000$ hit and $Q = 1,000,000$ miss lookups benchmark the contaminated table against an identical pristine *Clean Table* containing the exact same active keys without tombstones. Hit keys are sampled with replacement because $Q > N$. Table II reports the empirical metrics, and Fig. 4 illustrates the contamination timeline, probe surge, and latency profile.

The empirical data across all six checkpoints reveal four fundamental engineering and algorithmic insights:

1) **Slot Contamination Dynamics and Empty Slot Depletion**: As churn progresses, deletions repeatedly convert live slots into tombstones. Although subsequent insertions can reclaim tombstone slots, the uniform distribution of key hashes causes tombstones to disperse throughout the entire table. As seen in Fig. 4(a), the tombstone proportion escalates rapidly from 0.00% to 18.79% at $C = 0.25M$, 41.91% at $C = 1M$, and saturates at 49.96% ($523,876$ slots) at $C = 4M$. Concurrently, the proportion of pristine `EMPTY` slots undergoes exponential extinction, plummeting from 50.00% ($524,288$ slots) down to 8.09% at $C = 1M$, 1.36% at $C = 2M$, and reaching a critical nadir of only **0.04%** (412 remaining slots out of $1,048,576$) at $C = 4M$. At this stage, 99.92% of the capacity available outside active keys is contaminated by tombstones.
2) **Asymmetric Probe Divergence (Hit Plateau vs. Catastrophic Miss Explosion)**: The impact of tombstone saturation on search depth is severely asymmetric between query types:
   - *Successful Lookups (Hit Probes)*: Hit queries remain well-behaved, increasing from 1.500 to 2.346 probes (a $1.56\times$ increase). Because a successful lookup terminates immediately upon matching the target key, it only traverses tombstones that displaced the key from its initial hash bucket within its localized cluster.
   - *Unsuccessful Lookups (Miss Probes)*: In standard Linear Probing, an unsuccessful lookup must probe sequentially until encountering an `EMPTY` slot. When empty slots are depleted to 0.04%, miss searches cannot terminate early and are forced to traverse massive contiguous clusters of tombstones. As detailed in Table II and Fig. 4(b), miss probe depth explodes non-linearly: from 2.503 probes at $C = 0$ to 17.237 at $C = 1M$, 86.070 at $C = 2M$, and reaching **2,848.989 probes** at $C = 4M$. This is approximately a $1,138\times$ increase over the initial contaminated table and a **1,141.51**$\times$ increase over the Clean Table at $C = 4M$.
   - *Clean Table Immunity*: In contrast, the reference *Clean Table* containing the identical $524,288$ live keys maintains probe efficiency across all milestones: $1.496 - 1.502$ hit probes and $2.490 - 2.503$ miss probes, close to Knuth's classical theoretical bounds $\mathbb{E}[P_{\text{hit}}] \approx \frac{1}{2}(1 + \frac{1}{1-\alpha}) = 1.500$ and $\mathbb{E}[P_{\text{miss}}] \approx \frac{1}{2}(1 + \frac{1}{(1-\alpha)^2}) = 2.500$.
3) **Lookup Latency Kinetics**: The one-million-query miss batch grows from 48.05 ms at $C = 0$ to **18,223.54 ms** (18.22 seconds) at $C = 4M$. At the final milestone this is $203.87\times$ the Clean Table time of 89.39 ms. Hit-time measurements are noisy (for example, the Clean Table takes 131.00 ms at $C = 4M$ versus 34.93 ms

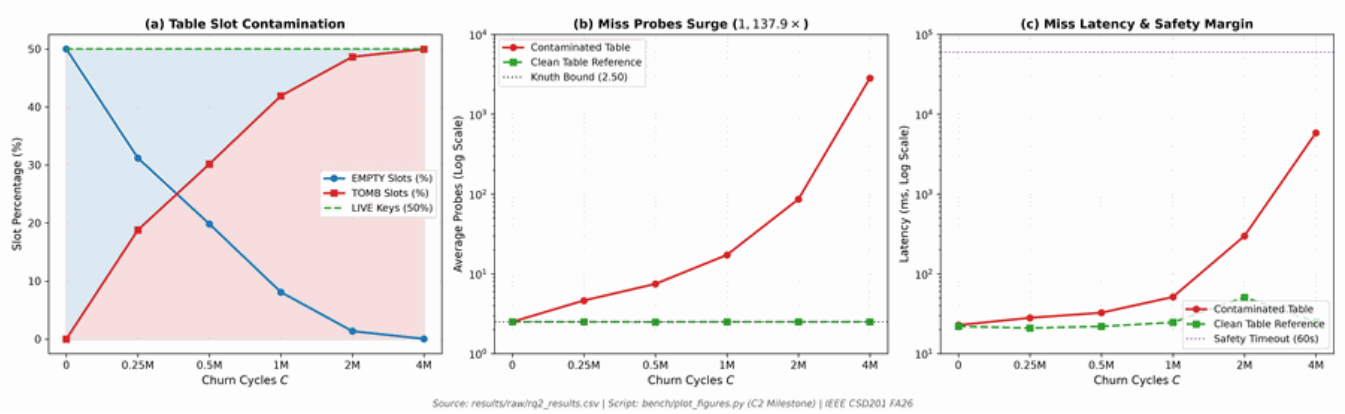


Fig. 4. Empirical evaluation of RQ2 using one-million-query batches: tombstone accumulation, probe explosion, and latency ($M = 2^{20}, \alpha = 0.50$): (a) Table slot contamination and extinction of empty slots; (b) Miss-probe divergence ($1,141.5\times$ vs. Clean Table at $C = 4M$); (c) Lookup-time divergence. The 60-second timeout applies to the complete JVM process, not only the plotted lookup phase.

TABLE II
EMPIRICAL COMPARISON: CONTAMINATED TABLE (BASE) VS. *Clean Table* ($M = 2^{20}, \alpha = 0.50, N = 524,288$)

Milestone C	TOMB %	EMPTY %	Hit Probes (Base vs Clean)	Miss Probes (Base vs Clean)	Hit (ms)	Miss (ms)	Miss-Probe Ra
$C = 0$	0.00%	50.00%	1.500 vs 1.500	2.503 vs 2.503	27.54 vs 34.27	48.05 vs 41.35	1.00×
$C = 0.25M$	18.79%	31.21%	1.852 vs 1.498	4.627 vs 2.501	37.22 vs 32.27	71.24 vs 49.37	1.85×
$C = 0.50M$	30.16%	19.84%	2.045 vs 1.496	7.472 vs 2.490	41.36 vs 34.74	69.76 vs 44.56	3.00×
$C = 1.00M$	41.91%	8.09%	2.230 vs 1.500	17.237 vs 2.501	49.29 vs 32.12	97.52 vs 44.85	6.89×
$C = 2.00M$	48.64%	1.36%	2.331 vs 1.502	86.070 vs 2.500	75.04 vs 34.93	650.60 vs 44.49	34.43×
$C = 4.00M$	49.96%	0.04%	2.346 vs 1.502	**2848.989** vs 2.496	53.18 vs 131.00	**18223.54** vs 89.39	**1,141.51**×

at $C = 2M$), so the probe-count trend provides the more stable mechanism-level evidence in this single-repeat campaign.

4) **Timeout Analysis, Asymptotic Transition to $O(M)$, and Periodic Rebuilding (τ)**: The automated runner applies a 60-second timeout to each complete JVM process, including startup, table construction, churn, and queries. The longest process in the corrected one-million-query campaign took 46.044 seconds (the `base`, $C = 4M$, miss-time run), leaving 13.956 seconds before the limit. Consequently, all 48 measured runs achieved the `ok` status with zero timeout exceptions. The CSV's `elapsed_ms` records lookup time only; it is not the full process duration.

   However, the trajectory reveals a dangerous asymptotic property: when $C \to \infty$ without rebuilding, the number of empty slots approaches zero (EMPTY $\to 0$). In that limit, every unsuccessful query is forced to scan all $M = 2^{20}$ slots in a full circular pass before failing, degrading per-query complexity from expected $\mathcal{O}(1)$ to deterministic $\mathcal{O}(M)$. In real-time or multi-threaded production workloads, the measured $203.87\times$ miss-latency surge at $C = 4M$ illustrates the resulting timeout risk.

   These results motivate **periodic rebuilding**: reconstructing the table whenever the tombstone count reaches $\tau \cdot M$ resets all tombstones to zero. RQ2 itself does not evaluate rebuilding; its effect is measured in RQ3, where $\tau = 0.20$ ($\alpha = 0.50$) and $\tau = 0.10$ ($\alpha = 0.80$) trigger 14 and 23 rebuilds over $4M$ churn cycles (about one rebuild per 3×10^5 and 1.8×10^5 cycles). Rebuilding bounds the degradation but does not remove it (4.22 vs. 2.50 miss probes at $C = 4M$, $\alpha = 0.50$), and since only two values of τ were tested, we make no claim about an optimal threshold.

C. RQ3: Mechanism Ablation of Robin Hood Hashing

The moderate-load campaign compares `lp-tomb` ($\tau = 0.2$), `lp-backshift`, `rh-noearly`, and `rh` at $M = 2^{20}$, $\alpha = 0.50$, and $C \in \{0, M/4, M/2, M, 2M, 4M\}$. For every checkpoint, three seeds are used, with one run per seed for each query and measurement mode. Table III reports the largest churn checkpoint ($C = 4M$): probe counts are means over the three seeds, and lookup times are medians of the three per-seed runs.

1) **Deletion policy**: compare tombstone-based LP with rebuilding against LP and Robin Hood backward-shift deletion.
2) **DIB-based insertion**: compare `rh-noearly` with `lp-backshift` while disabling Robin Hood's DIB-based lookup termination.
3) **Early-exit lookup**: compare `rh` with `rh-noearly` on unsuccessful queries.

Figures 5 and 6 show the probe-count and lookup-time trends across the six checkpoints; Fig. 7 summarizes mean probes across checkpoints.

At $C = 0$, the three variants without Robin Hood early exit begin with the same mean miss count (2.499 probes); `rh` already averages 1.750 probes because its early-exit rule is

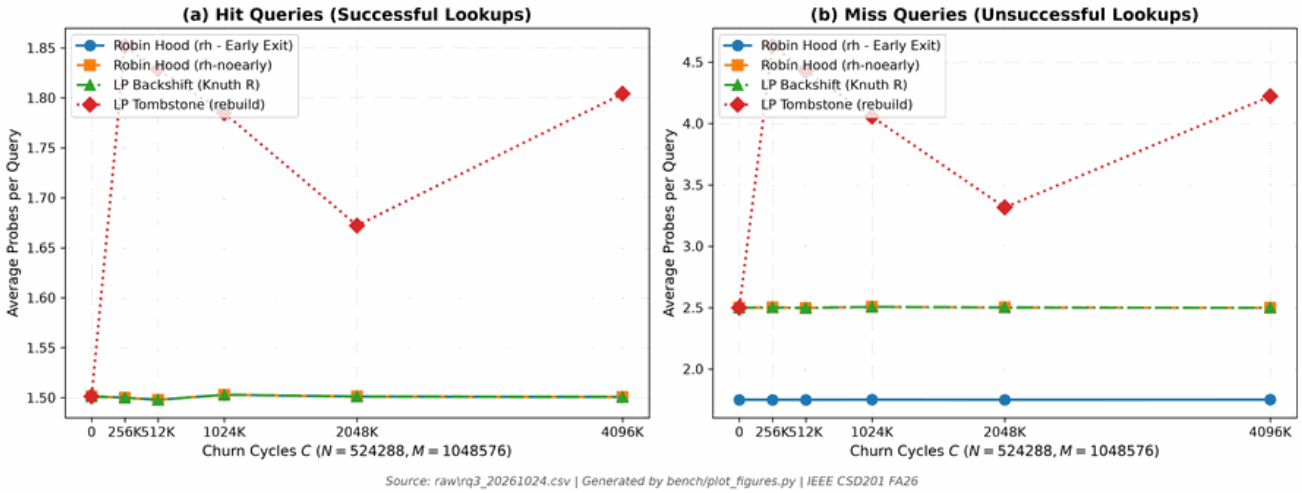


Fig. 5. Mean hit and miss probes by churn checkpoint for the four RQ3 implementations at moderate load ($\alpha = 0.50, M = 2^{20}$).

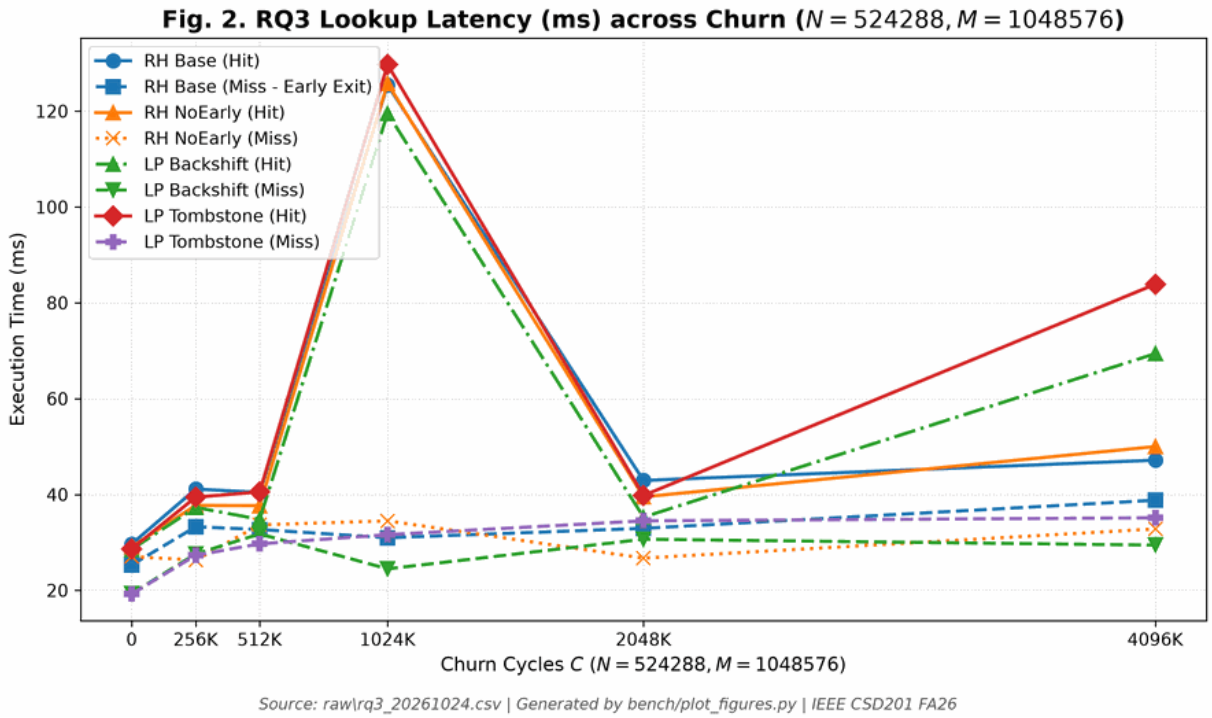


Fig. 6. Lookup-time measurements across RQ3 churn checkpoints at moderate load. Each point summarizes three per-seed runs.

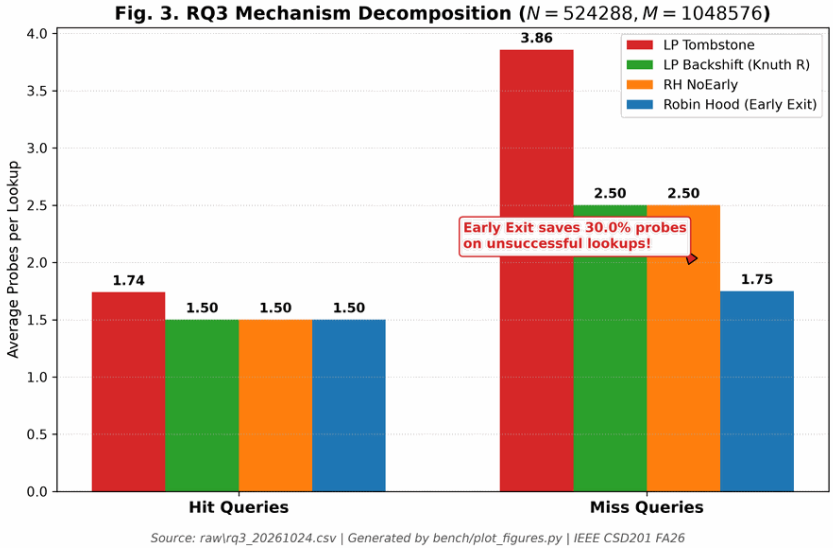


Fig. 7. Mean probes averaged across the RQ3 churn checkpoints and seeds at moderate load ($\alpha = 0.50$).

TABLE III
RQ3 RESULTS AT $C = 4M$ ($M = 2^{20}$, $\alpha = 0.50$). PROBE MEANS AND MEDIAN LOOKUP TIME OVER THREE SEEDS.

Algorithm	Hit probes	Miss probes	Hit (ms)	Miss (ms)
`lp-tomb` ($\tau = 0.2$)	1.804	4.220	83.89	35.14
`lp-backshift`	1.501	2.499	69.40	29.44
`rh-noearly`	1.501	2.499	50.01	32.82
`rh`	1.501	**1.751**	47.17	38.81

active. `lp-tomb` with rebuilding then shows a non-monotone miss-probe trend across churn checkpoints, reaching 4.220 at $4M$. At that checkpoint, `lp-backshift` and `rh-noearly` both average 2.499 miss probes, while `rh` averages 1.751. Thus, early exit reduces mean miss probes by approximately 30.0% relative to `rh-noearly` for this workload. Hit probes remain close to 1.501 for the three non-tombstone implementations; the `lp-tomb` mean is 1.804.

1) High-Load Stress Campaign ($\alpha = 0.80, \tau = 0.10$): We also ran a high-load stress campaign, evaluating all four implementations under $\alpha = 0.80$ ($N = 838,861, M = 2^{20} = 1,048,576$) across the same six churn milestones up to $C = 4M$. For `lp-tomb`, the periodic rebuild threshold is configured to $\tau = 0.10$.

TABLE IV
RQ3 HIGH-LOAD RESULTS AT $C = 4M$ ($M = 2^{20}$, $\alpha = 0.80, \tau = 0.10$). PROBE MEANS AND MEDIAN LOOKUP TIME OVER THREE SEEDS.

Algorithm	Hit probes	Miss probes	Hit (ms)	Miss (ms)
`lp-tomb` ($\tau = 0.10$)	3.162	14.605	58.89	58.67
`lp-backshift`	2.998	12.933	51.91	35.45
`rh-noearly`	2.998	12.933	67.47	52.80
`rh` (Full)	2.998	**3.398**	65.19	**43.38**

The high-load empirical results (Table IV and Fig. 8) reveal crucial architectural insights:

1) **Decisive Superiority of Early Exit under Load**: While at moderate load ($\alpha = 0.50$) early exit saves $\approx 30.0\%$ of miss probes, under heavy load ($\alpha = 0.80$) its benefit surges dramatically. Unsuccessful queries in `rh` average only **3.400 probes** across all churn cycles, compared to **12.982 probes** in `lp-backshift` and `rh-noearly`—a **73.8% reduction** (73.7% at $C = 4M$: 12.933 vs. 3.398). Compared to tombstone-based linear probing (`lp-tomb`, which averages 20.844 miss probes and peaks at 22.263 at $C = 2M$), Robin Hood achieves an **83.7% reduction** in lookup probe depth.
2) **Tombstone Pollution Escalation**: Even with aggressive periodic rebuilds at $\tau = 0.10$, tombstone accumulation in `lp-tomb` inflates both hit probes (to a mean of 3.652 over the checkpoints) and miss probes (to a mean of 20.844). In contrast, backward-shift deletion in `lp-backshift` and `rh` completely prevents tombstone formation (0 tombstones maintained throughout $4M$ operations), closely matching Knuth's theoretical hit bound $\frac{1}{2}(1 + \frac{1}{1-\alpha}) = 3.000$ and miss bound $\frac{1}{2}(1 + \frac{1}{(1-\alpha)^2}) = 13.000$.
3) **Latency Characteristics**: At $C = 4M$, the median miss-lookup time of `rh` (43.38 ms) is lower than that of `lp-tomb` (58.67 ms) and `rh-noearly` (52.80 ms) but higher than that of `lp-backshift` (35.45 ms), and hit-lookup times show no consistent ordering (Table IV). The large probe reduction therefore does not translate into a uniformly lower lookup time in our measurements.

The lookup-time medians do not follow probe counts uniformly across all regimes: for example, at $\alpha = 0.50$, `rh` has fewer miss probes but a higher miss-time median than `rh-noearly` in this campaign. With one repeat per seed, these timing results are descriptive and do not establish an absolute latency advantage across all loads. Under stress conditions ($\alpha = 0.80$) the probe-count reduction is large, but

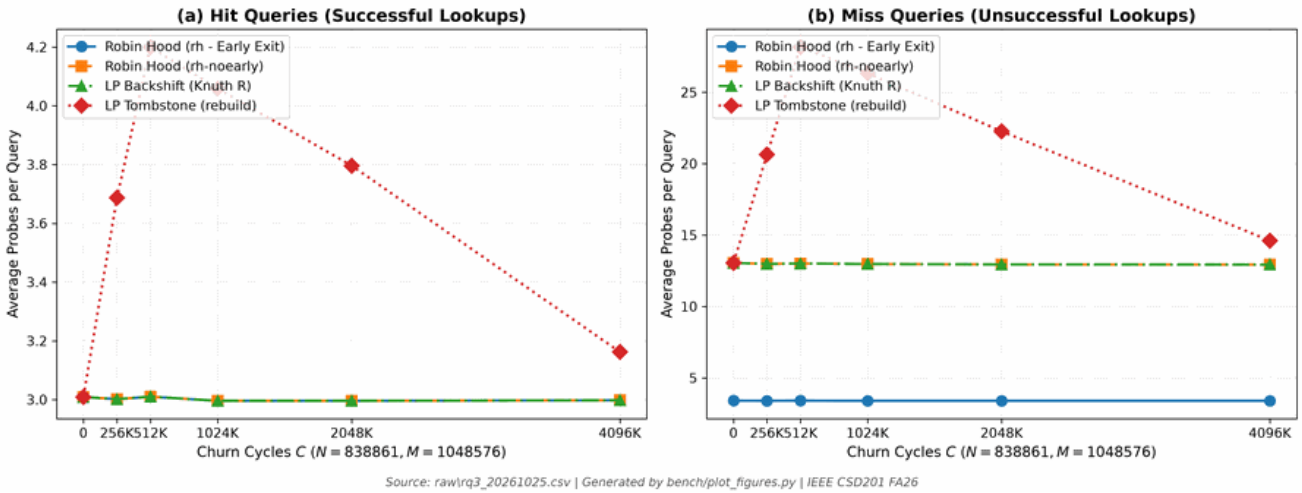


Fig. 8. Mean hit and miss probes across churn checkpoints under high load ($\alpha = 0.80, M = 2^{20}$). Early Exit caps miss probes at ≈ 3.40 regardless of churn, while `lp-tomb` surges past 20 probes.

its effect on elapsed time is not uniform. The lookup timing excludes table construction and churn, which are recorded separately as build time.

2) Workload Block Profiling and Mechanism Ablation : To answer RQ3's core question regarding the precise quantitative contribution of each algorithmic mechanism and to capture operational jitter invisible to aggregate averages, we performed micro-profiling in blocks of 10^4 operations across the entire $C = 4M$ churn workload (8.39×10^6 operations). Table V details the runtime characteristics and internal structural state of the tables at completion.

TABLE V
WORKLOAD CHURN ($C = 4M$, 10^4 OPS/BLOCK) AND TABLE STATE AT COMPLETION ($M = 2^{20}$)

α	Algorithm	Total (s)	Med (ms)	Max (ms)	Rebuilds	Max DIB
0.50	`lp-tomb` ($\tau = 0.20$)	2.25	1.88	91.46	14	45
0.50	`lp-backshift`	1.99	1.77	121.48	**0**	41
0.50	`rh-noearly`	2.12	2.05	106.76	**0**	**9**
0.50	`rh` (Full)	2.20	2.09	92.07	**0**	**9**
0.80	`lp-tomb` ($\tau = 0.10$)	2.55	2.19	88.69	23	220
0.80	`lp-backshift`	2.60	2.58	113.94	**0**	195
0.80	`rh-noearly`	2.79	2.79	116.51	**0**	**25**
0.80	`rh` (Full)	2.72	2.70	108.80	**0**	**25**

In terms of total workload time, no implementation dominates: `lp-backshift` is fastest at $\alpha = 0.50$ (1.99 s) and `lp-tomb` at $\alpha = 0.80$ (2.55 s), with `rh` about 10.6% and 6.7% slower, respectively. The Max column also shows isolated outlier blocks of roughly 90–120 ms in every implementation, including those that never rebuild; these exceed the rebuild blocks themselves and are not diagnosed here (GC or OS scheduling are possible causes), so Max block time alone does not isolate rebuild cost.

The fine-grained telemetry and mechanism ablation (Fig. 9 and Fig. 10) isolate the exact engineering value of each design choice:

1) **Mechanism 1: Eliminating Tombstones via Backward-Shift Deletion (`lp-tomb` vs `lp-backshift`)**: Under moderate load ($\alpha = 0.50$), `lp-tomb` triggers 14 full table rebuilds, and under high load ($\alpha = 0.80$), it triggers 23 rebuilds. While normal blocks execute in ≈ 1.87 ms ($\alpha = 0.50$) and ≈ 2.18 ms ($\alpha = 0.80$), rebuild blocks experience severe latency spikes, averaging 10.14 ms and 13.34 ms (a $5.4\times$ **to** $6.1\times$ **surge**), peaking at 36.10 ms ($\alpha = 0.50$) and 29.6 ms ($\alpha = 0.80$). By replacing tombstones with Knuth Algorithm R backward shift, `lp-backshift` eliminates all rebuild pauses (0 rebuilds) and maintains zero tombstones; isolated outlier blocks nevertheless occur in every implementation (Table V).
2) **Mechanism 2: Variance Reduction via Robin Hood DIB Swapping (`lp-backshift` vs `rh-noearly`)**: Both algorithms eliminate tombstones, yet their internal displacement distributions diverge drastically. In `lp-backshift`, clusters form unchecked, resulting in heavy-tailed displacement where the maximum probe distance (Max DIB) reaches 41 at $\alpha = 0.50$ and escalates to 195 at $\alpha = 0.80$ (peaking at 420 during churn). In contrast, Robin Hood's rich-to-poor swapping equalizes probe distances, truncating Max DIB down to **9** at $\alpha = 0.50$ (a **78.0% reduction**) and **25** at $\alpha = 0.80$ (an **87.2% reduction**), tightly bounding tail latency.
3) **Mechanism 3: Search Acceleration via Early Exit (`rh-noearly` vs `rh`)**: Early exit terminates lookups as soon as the probing distance exceeds the slot's stored DIB ($DIB_{probe} > DIB_{slot}$). At moderate load ($\alpha = 0.50$), this cuts mean unsuccessful probe depth from 2.499 to 1.751 (30.0% reduction). Under heavy load ($\alpha = 0.80$), its impact is dramatic: miss probes drop from 12.933 to **3.398**—a **73.7% reduction** in lookup

depth at $C = 4M$.

VI. Threats to Validity

Several experimental factors and methodological constraints could influence the validity of our findings:

1) **JIT Compilation and JVM Warmup**: Benchmarking in Java is susceptible to HotSpot JIT compilation tiers and dynamic deoptimization. In all research questions (RQ1–RQ3), each configuration is launched in an isolated JVM process to prevent class-loading contamination. To mitigate cold-start distortion while avoiding code mutation, small warmup passes (1,000 queries) precede timed lookups, and dead-code elimination (DCE) is countered via an accumulated 64-bit checksum (`output_check`). Furthermore, probe counting is conducted in a completely separate `count` execution pass, ensuring probe depth measurements remain mathematically deterministic and immune to JIT jitter. JIT optimizations may also favor contiguous primitive-array access in Linear Probing over pointer-based node traversal in Separate Chaining; we did not inspect the compiled code.
2) **Garbage Collection (GC) Interference and Memory Telemetry**: Workloads with extensive object allocation can trigger unpredictable stop-the-world GC pauses. In open-addressing tables (`lp-tomb`, `lp-backshift`, `rh`), implementations use flat primitive arrays; in contrast, Separate Chaining (`sc` in RQ1) allocates individual `Node` instances. Peak heap usage is tracked via Java's `MemoryPoolMXBean`, but `heap_peak_bytes` covers the entire benchmark process, including workload bookkeeping such as the active-key array and the `seenKeys` set, as well as table and query data; it is not a table-only memory measurement. Explicit GC and peak-counter reset occur before table construction, not immediately before lookup timing, so peak telemetry should be interpreted as process-level high-water usage rather than an isolated table footprint.
3) **Execution Timeout Safety and Asymptotic Boundary**: Each benchmark trial is governed by a strict 60-second process timeout (60,000 ms). In the corrected RQ2 campaign, the longest complete process took 46.044 seconds at $C = 4M$ for the contaminated-table miss-time run; all 48 processes completed without timeout. The query-only miss latency for that run was 18.224 seconds and must not be mistaken for total process time. As EMPTY $\to 0$ asymptotically as $C \to \infty$, unsuccessful lookups approach exhaustive $\mathcal{O}(M)$ scans; workloads may exceed a timeout depending on table size and query volume.
4) **Machine and Hardware Workload Isolation**: All official measurements were performed on a single dedicated machine (11th Gen Intel Core i5-11500H @ 2.90 GHz, 32 GB RAM, Windows 11 Pro) with background user processes terminated and network services suspended. Running all algorithms under identical hardware configurations ensures meaningful cross-algorithmic comparisons.
5) **Hash Function, Seed Diversity, and Timing Variance**: All implementations utilize the standard 64-bit Fibonacci hashing function $h(k) = (k \cdot \phi) \ggg (64 - p)$ with $\phi =$ `0x9E3779B97F4A7C15` (unsigned value 11400714819323198485). RQ3 uses multiple seeds $(42, 1337, 20261019)$; RQ2 currently uses only seed 42 and one repeat, so its results do not estimate between-seed or between-repeat variability and should be interpreted as a controlled single-seed case study, not a median across runs. Timed configurations use one 1,000-query warmup pass and one measured pass per JVM; this reduces but does not eliminate JIT tiering and process-level timing variation. Future RQ2 replications with additional seeds and repeats are needed to quantify that uncertainty. Matching `output_check` values validate lookup results but do not validate timing stability.

VII. Conclusion

This empirical study systematically evaluates collision resolution strategies in open-addressing and chained hash tables under high-churn transactional workloads. By comparing Separate Chaining, Linear Probing with tombstones, Linear Probing with Knuth Algorithm R backshift deletion, and Robin Hood hashing, we provide concrete engineering recommendations:

- **Separate Chaining vs. Linear Probing (RQ1)**: Algorithmic probe count does not translate directly to lower execution latency in Java. While Separate Chaining maintains sub-2 probe counts under high load factors ($\alpha \geq 0.90$) and superior immunity to clustered hashes, it incurs a memory penalty of roughly $1.5\times$–$2.7\times$ (depending on α) and pointer-chasing cache misses. Linear Probing delivers superior raw lookup throughput at moderate loads ($\alpha \leq 0.75$) due to flat primitive array spatial locality and CPU stream prefetching.
- **Tombstone Pollution and Rebuilding (RQ2)**: In the measured single-seed workload with continuous deletions, tombstone accumulation depletes empty slots to 0.04% and raises miss depth to $2,848.99$ probes at $C = 4M$. The one-million-query miss batch takes 18.22 seconds on the contaminated table versus 89.39 ms on the Clean Table. RQ3 shows that periodic rebuilding bounds but does not remove this degradation (4.22 vs. 2.50 miss probes at $\alpha = 0.50$, $C = 4M$), whereas Knuth's Algorithm R (`lp-backshift`) removes tombstones altogether.
- **Robin Hood Hashing and Mechanism Ablation (RQ3)**: In the measured RQ3 workloads, Robin Hood hashing with early exit reduces mean unsuccessful-lookup probes by $\approx 30.0\%$ at moderate load ($\alpha = 0.50$) and by a dramatic **73.8%** under heavy load ($\alpha = 0.80$, dropping from 12.98 down to 3.40 probes) compared to `rh-noearly` and `lp-backshift`, while cutting

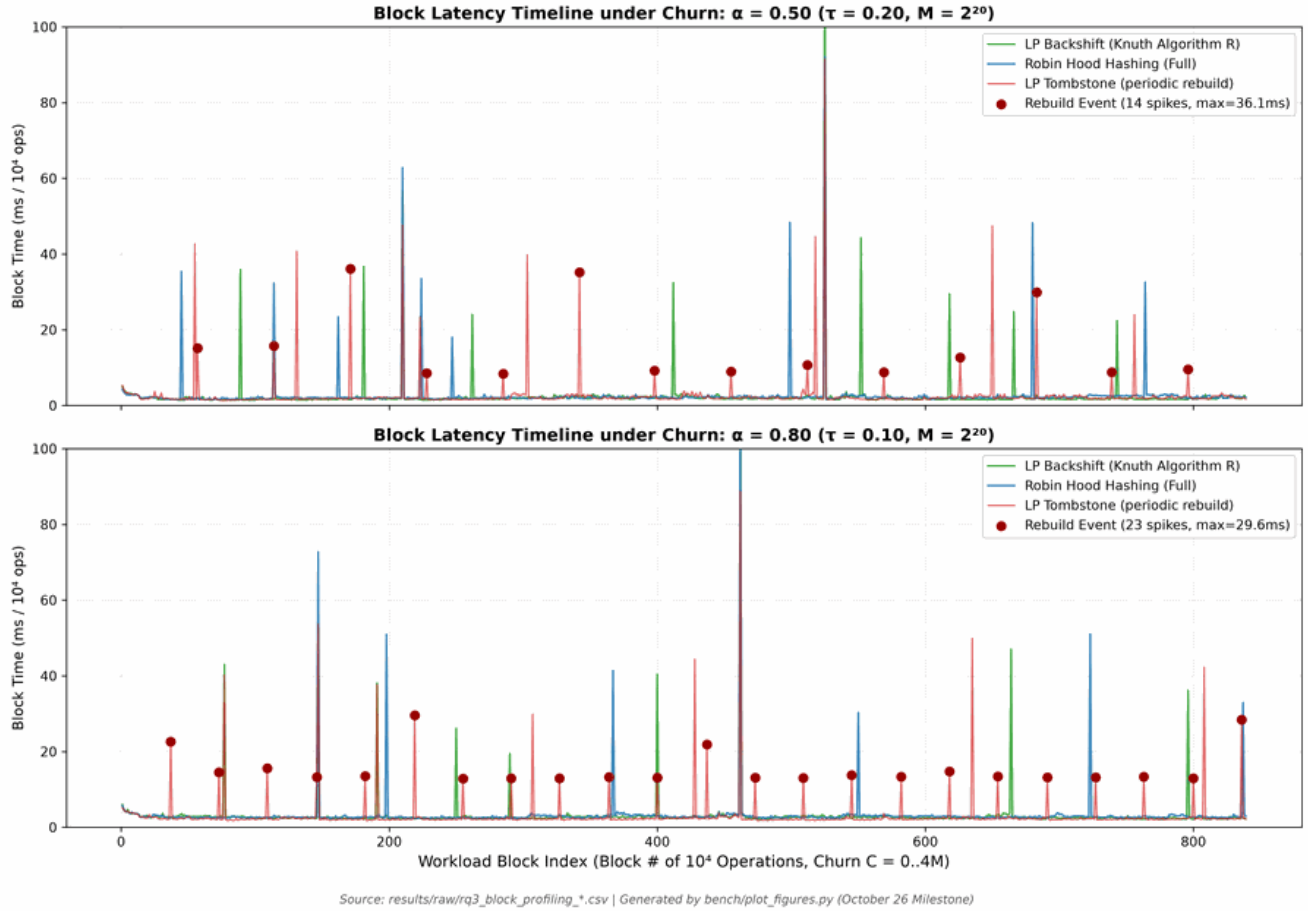


Fig. 9. Workload block latency timeline (10^4 operations/block) across $C = 4M$ churn cycles. Red circles indicate periodic table reconstruction in `lp-tomb` ($5.4\times$ to $6.1\times$ latency spikes), absent in `lp-backshift` and `rh`.

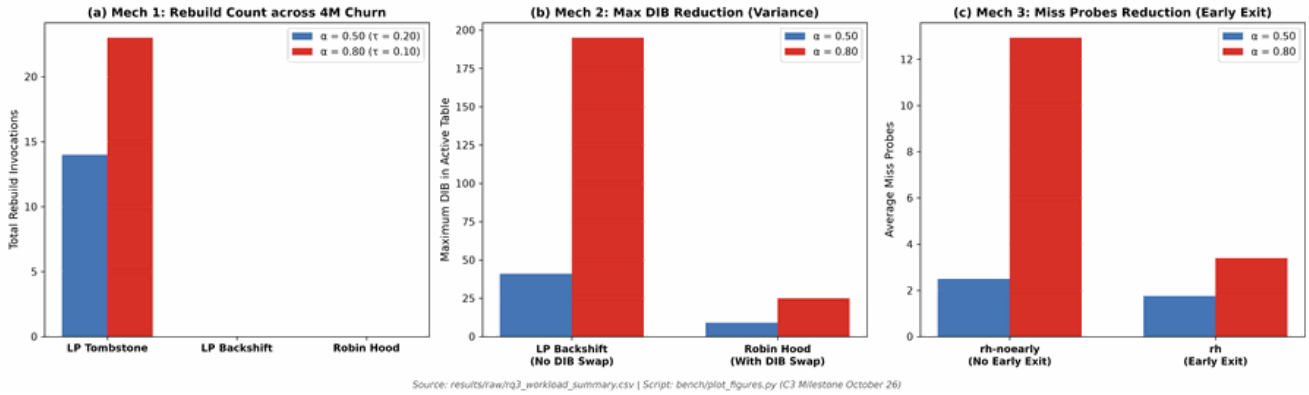


Fig. 10. Quantitative decomposition of Robin Hood hashing's three core mechanisms: (a) eliminating rebuild events, (b) compressing maximum probe distance (DIB variance), and (c) slashing unsuccessful lookup depth via early exit.

miss probes by **83.7%** relative to the checkpoint-averaged `lp-tomb` value at $\alpha = 0.80$. Elapsed lookup and total workload times do not show a uniform advantage for Robin Hood, so the benefit established here lies in probe count and tail displacement (Max DIB), which matters most for workloads dominated by absent-key queries.

ACKNOWLEDGMENT

In accordance with Section 7.7 of the CSD201 project guidelines, the authors declare that AI assistance (large language model tools) was utilized strictly for drafting LaTeX formatting templates, grammatical refinement, and bibliographic structuring under strict human review. All algorithmic implementations, benchmark designs, empirical measurements, data integrity checks, and scientific analyses were conceived, verified, and conducted by the student authors.

REFERENCES

[1] D. E. Knuth, *The Art of Computer Programming, Vol. 3: Sorting and Searching*, 2nd ed. Reading, MA, USA: Addison-Wesley, 1998, §6.4.

[2] W. W. Peterson, "Addressing for random-access storage," *IBM J. Res. Dev.*, vol. 1, no. 2, pp. 130–146, 1957.

[3] P. Celis, P.-Å. Larson, and J. I. Munro, "Robin Hood hashing," in *Proc. 26th Annu. Symp. Found. Comput. Sci. (FOCS)*, 1985, pp. 281–288.

[4] R. Sedgewick and K. Wayne, *Algorithms*, 4th ed. Boston, MA, USA: Addison-Wesley, 2011, §3.4.